2024 International Symposium of Systems, Advanced Technologies and Knowledge (ISSATK) | 979-8-3315-4017-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISSATK62463.2024.10808950

Fog Computing Architecture for an IoT enabled smart parking system

Faten BEN ABDALLAH
Automatic Research Laboratory LARA
National Engineering School of Tunis ENIT
University of Tunis El Manar
Tunis, Tunisia
faten.benabdallah@enit.utm.tn

Ines Belkhiria
National Engineering School of Sousse ENISo
University of Sousse
Sousse, Tunisia
ines.belkhiria@gmail.com

***Abstract*—In recent years, the benefits of cloud computing have placed it at the heart of the architectures proposed for the Internet of Things (IoT). The homogeneous, predictable, and efficient infrastructure has made Cloud an adequate solution for the processing and analysis of data from IoT objects. However, the benefits of using cloud proved to be problematic for latency sensitive IoT systems that require geographic distribution, consideration of the local environment, and mobility of objects. Fog computing is an emergent concept aimed at extending cloud at the edge of the IoT. Thus, it envisions a layer of nodes (Fogs), making it possible to provide connected objects with support for communication management and data persistence.**

This paper is motivated by the promising opportunities for the concept of fog computing. We propose a fog computing Framework based on SSN ontology for IoT applications, by applying it to an intelligent parking system that, on the one hand, exploits IoT objects with services and, on the other hand, increases the representation and data consumption with local analyzes at the Fog nodes.

For demonstrating, the effectiveness of our implemented framework, simulations have been performed in SUMO, a simulation tool used for urban traffic. Simulation results exhibit that the proposed fog-based implementation reduces the parameters of latency and network usage when contrasted with cloud-based deployment of the smart car parking system.

Keywords: Internet of Things (IoT), Cloud Computing, Fog Computing, smart parking.

I. Introduction

Due to the increased population and limited spatial resources of the cities, finding the best location for vehicle parking in the parking area is becoming one of the major problems and the time and gasoline consuming process in almost every country. Therefore, smart parking systems, designed to aid in locating available parking spots upon vehicle entry, have been developed and implemented across numerous parking facilities. Several researchers have put forward smart car parking systems that utilize cloud computing and the Internet of Things (IoT). For this purpose, substantial numbers of sensors are necessary to collect data and ensure precise service provision, albeit at significant costs. The current architectural model and trends in IOT are turning to cloud-centric architectures. The processing and analysis of data from objects connected to the IOT occurs only in the cloud, posing issues related to network bandwidth, communication latency, and the ability to access local information.

Cloud computing has limitations, particularly as data transmission rates increase, resulting in longer response times and higher service delays for end-users due to the influx of data. This high volume of data transmission over the network raises the likelihood of errors and delays, including packet loss and transmission latency, which directly correlate with the amount of data sent through IoT devices to the cloud.

To address these challenges, fog computing emerges as a suitable solution. Positioned between IoT devices and the cloud, the fog computing layer offers storage, preprocessing, execution, networking, and computational services. By operating at the network edge, closer to end-users, fog computing enhances efficiency and reduces latency compared to traditional cloud-based approaches [1].

We anticipate the aggregation of detection activities and the distribution of collaborative interaction between connected objects at the edge of the IOT network as opportunities to tackle the aforementioned challenges. Therefore, we propose to adapt and extend the idea of the Fog Computing paradigm to embrace a distributed computing model for IOT. Such a model will help to use resources more efficiently and support more sophisticated application scenarios. Fog Computing aims to reduce the implications of the cloud by filtering the data produced by the growing number of sensors and other IOT devices connected to the Fog Computing environment. This better use of resources at the edge speeds up communication and task execution. However, the development, implementation, maintenance of IOT applications in such an environment in an efficient manner are major research challenges. In this paper we have focused on providing an architecture of a Fog Computing Framework based on SSN ontology and application for an intelligent parking system that, on the one hand, operates IoT objects with services; and on the other, increases the representation and consumption of data with local analyzes at the Fog node level.

The remainder of this paper is structured as follows: Section II presents an overview of recent research on smart car parking systems. Section III presents the proposed fog-based framework whereas implementation setup is discussed in Section

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.

IV. Section V presents the simulation results and discussion. Section VI concludes the paper and also delineates future research directions.

II. RELATED WORKS

Several studies have already been performed on smart vehicle parking allocation and management using the fog computing concept.

Balfaqih et al. [2] proposed a multi-layer architecture comprising multiple sensor nodes, along with fog and cloud computing layers to overcome the limitations of single gateway-centered solutions. Moreover, the authors presented a parking-spot selection algorithm for determining the optimal parking spot with the shortest arrival time. Computation latency and network delay are not discussed by the authors for the cloud environment.

Tandon et al. [3] used an online reservation and a modified greedy algorithm for the allocation of parking areas to the vehicles. Fog-nodes keep track of all the requests made by vehicle owners and calculate the total time cost of the vehicles. There is no discussion about network and computation delay.

Vehicular Fog Computing (VFC) provides low-latency service to vehicles and end users by extending fog computing to the conventional vehicular networks. In fact, roadside units (RSUs) integrate fog computing servers to deliver communication and computation services to mobile terminals. Zhang et al. [4] investigated the potential of VFC combined with PVA (Parked Vehicle Assistance). PVA aims to enhance resource efficiency, particularly in terms of network connectivity. The proposed system provides a fog-aware smart parking reservation auction to attract the on-the-move vehicles to the available parking places by monetary rewards for service offloading. The offload price was improved through an iterative update process in a multi-round setting by transferring the CPU resources of the parked vehicles to other hotspots so that the total profit of the fog network is increased. The transmission delay is not calculated.

Awaisi et al. [5] proposed a fog-based smart parking architecture leveraging computer vision techniques for identifying vacant parking spaces.

Tang et al. [6] leverage the combination of fog computing and VANETs to provide customized parking services.

Kumara et al. [7] utilized hybrid Markov system in various parking scenarios, including shopping centers, airports, and residential areas, to determine each edge gadget effectively, which reduces latency and increases throughput.

Intelligence is moving to the edge of the network with many rising challenges and opportunities. We intend to bring collaboration to the edge of the network following the vision of fog computing. First, by providing a middleware to abstract the underlying heterogeneity based on the success of previous works, and second, by modeling the aggregation of thing services in order to support more elaborated collaborative applications.

III. MODEL ARCHITECTURE

The proposed smart parking system enhances the convenience and efficiency of vehicle parking by guiding drivers to available spots, reducing congestion and travel time. The architecture comprises three layers: an IoT parking layer, a fog layer, and a cloud layer (see Figure 1). The IoT parking layer includes sensor nodes that monitor parking lots, detect vehicles, and gather area-specific data. The fog layer consists of multiple fog nodes, each handling specific services based on their requirements and capable of processing multiple sensor nodes simultaneously. Finally, the cloud layer facilitates secure, real-time access to parking reservation information and directions via a web-based parking platform or mobile application.

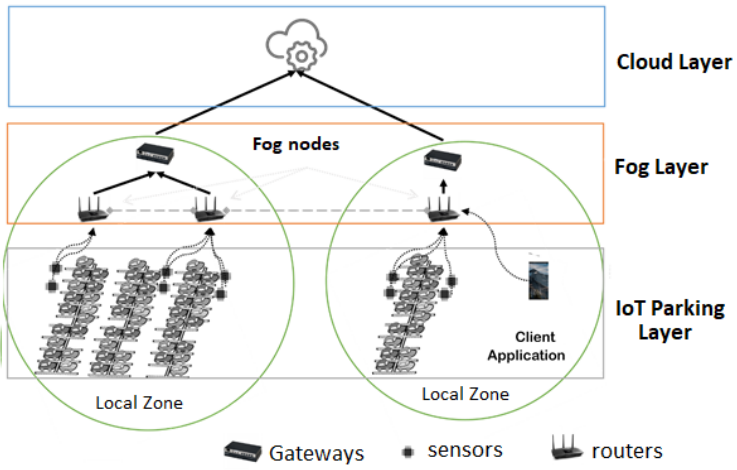


Figure 1. The proposed smart parking system architecture

The major fence of our smart parking system is the question of how to distribute IoT services/data on available fog resources. Therefore, inspired by the SSN ontology, we are proposing a framework to resolve this challenge in the fog layer.

In fact, ontologies provide a common vocabulary in a given domain and allow defining, with different levels of formality, the meaning of terms and the relationships between them. The huge number of sensors that are and will be deployed in the IoT dictates the need for an abstract information model to describe the heterogeneity of these devices. The Semantic Sensor Network Ontology, popularly known as SSN, was developed by an Incubator Group of the World Wide Web Consortium (W3C) over 2009 to 2011 [8]. SSN is an ontology used to describe sensors and sensor networks. The definition of such an ontology makes it possible to exploit the advantages of semantic technologies, for example, the integration of linked open data applications based on semantic web technologies.

A. *Semantic level:*

The proposed semantic ontology with spatio-temporal annotations and the semantics of user profiles, was developed from the ontology SSN, which complies with sensor standards currently adapted by the public. Some extensions and

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.

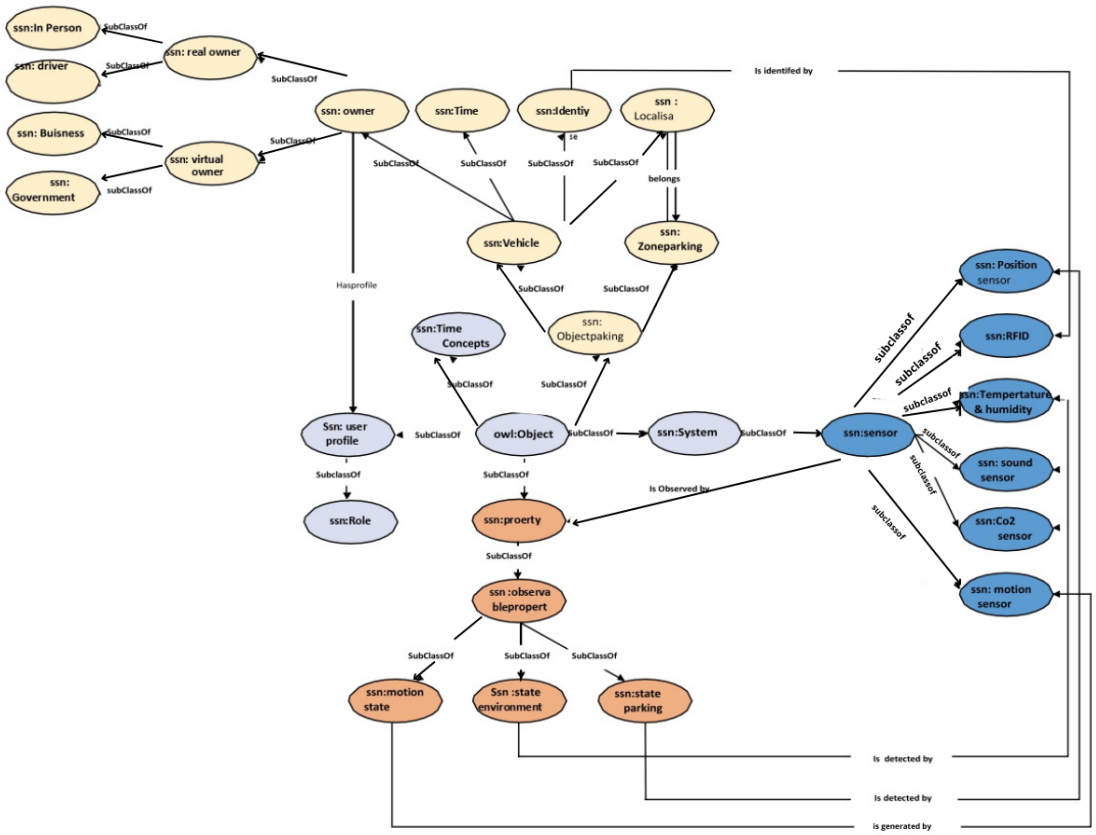


Figure 2. The RDF graph of the proposed ontology for smart parking systems

improvements are made, such as detailed types of sensors and the relationships between sensors and observable properties. The basic concepts involved in our framework for smart parking systems are shown in the RDF (Ressource Description Framework) graph illustrated in Figure 2 in which the sensors and observations are shown separately in ovals. RDF is a graph data model created by W3C, intended to provide a universal standard for information available on the web and make different sources easier to merge with each other.

6 types of sensors are mentioned, namely humidity, temperature, sound, and Co2 sensors for environmental factors, the motion sensor to detect any movement entering or leaving the car park, the position sensor for detecting state parking; and RFID for vehicle identification. Compared to other sensors, motion sensors are distinct in that their measurements are activated when motion events are triggered at the locations of detailed values. During this time, the observable property is mapped to corresponding metrics such as environmental factors, state of motion, and parking lot condition. A disadvantage of the SSN ontology is that it lacks the related spatial and temporal information sensors and sensor data. There is only one space reserved in SSN: observation for recording spatio-temporal information without entering further details. Therefore, in addition to the main concepts of SSN ontology, space-time annotations and user profiles are added to our proposed semantic model. The semantic ontology focuses mainly on the concept of location (SSN: location) ans belongs to the class SSN: Parking Zones. The SSN parking zone: ZonesParking is a subclass of parking objects, that is to say the locations are broken down into parking zones, each zone with sensors that are responsible for detecting its relative properties. Time information plays an important role in creating a comprehensive knowledge representation for smart parking. When sensor events are triggered in smart parking lots, the sensor data is accompanied by timestamps indicating when the event is docked. In our proposed semantic ontology, the temporal concepts mainly consist of temporal granularity (ssn: time concepts), which records the timestamps of events anchored by the sensors, and it has a granularity such as ssn:Day, ssn: Hour, ssn: Minute, etc.

B. Service level:

The services of our platform are spread over several Fogs nodes and IoT devices, each service defines a few simple interfaces that make it available for other services. Any service can be a client or consumer of other services. The use of a service-oriented architecture provides mechanisms to link

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.

the available services and create new services. The main services of our architecture and their interactions are detailed as follows:

1) Discovery service: This component is responsible for advertising, discovery, and registration of services. All services from all platforms and devices can be participating, registered and discovered using discovery services. This approach is used to allow applications and services in a Fog node to use available services and resources and to provide low latency responses. Consequently, the time required to discover the services is reduced as a minimum to use the services efficiently and precisely.
2) Gateway service: This component is responsible for load balancing between the various components of our architecture. Primarily, this component allows us to unify all of our services by sitting in front of them and acting as a proxy. The gateway service intercepts all requests and then routes them to the appropriate service. It mainly has four types of filters that allow traffic to be intercepted in different time lines of request processing for a particular transaction. pre-filters: are invoked before routing the request. post-filters: are invoked after routing the request. Route filters: are used to route the request. error filters: are invoked when an error occurs while processing the request.
3) Authentication and Authorization service: This component acts as a centralized point of authorization that allows applications and HTTP endpoints to identify the functionality of our application. It defines the following server side roles:
   - Resource owner: is responsible for controlling access to resources.
   - Resource server: provides an access token for clients to access resource server HTTP endpoints. It contains the point terminator, static resources, and dynamic web pages.
   - Authorization server: acts as an intermediary between the client and the owner of the resource, it manages the processing authorization process Resources.
4) Reservation Service: An online reservation for vehicle parking has been provided in the proposed template. The user must register to reserve vehicle parking online. Once the vehicle owner has completed the registration process, they can make a reservation by choosing the parking lot, estimated time of entry, and time of exit. Before the scheduled time, he can modify the parking reservation or cancel it. The system assigns it the nearest available parking zone, Before the end of the parking time, it can extend the parking period as well. If a user enters the car park without a reservation to park his vehicle at this time and uses his mobile device for identification. He taps on the mobile device, and the device will assign him to the nearest available parking space. The history of online reservation information is also updated in the cloud so that other vehicle owners who have requested parking can be allocated space in the reserved parking lot. This parking information should be synchronized with the parking model of the Fog environment for smart parking purposes. The online process for booking vehicle parking is followed using Fog nodes. These update the availability of parking space in the Clouds. The owner of the vehicle, whose parking space is reserved in advance, must reach the parking lot in good time, otherwise, this parking lot will be allocated to another. The online parking system for vehicle reservations is more advantageous compared to the on-site parking system, which is still used in our proposed model. Fog nodes were used to store and manage the settings below and the status of the parking lot. We based our strategy for the attribution of the parking area for vehicles on the modified Greedy algorithm proposed by Gupta et al. [3] : Fog-nodes keep track of all the requests made by vehicle owners. After receiving the parking space request, the availability will be checked; if the space is available for parking, then the special request for parking is processed, and the location is assigned to the vehicle accordingly. If no parking space is available at that particular time, the request will be redirected to another parking lot.
5) System Management: It is the component responsible for managing, updating critical information on IOT devices and providing its information for other services, which ensures system consistency is maintained. The semantic layer acts as a middleware built as a service-oriented architecture based on IOT objects. This layer is designed to answer the big questions of heterogeneity of sensor data.

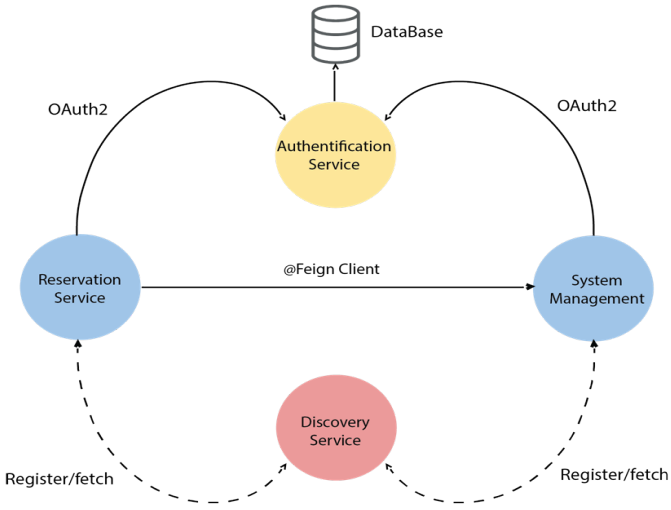


Figure 3. Interactions between the authentication service and other services

IV. IMPLEMENTATION

To evaluate the proposed smart parking model, we used the traffic Simulation of Urban Mobility (SUMO) simulator

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.

[9]. SUMO, an open-source urban traffic simulation toolset, includes a suite of applications designed to simulate traffic environments with various flows, aiming to closely resemble real-world scenarios.

The ontologies are managed using Apache Jena, an open-source framework for handling RDF graphs. This allows for seamless reading, writing, and interaction with data. The system relies on the knowledge defined in the ontology, while remaining flexible for incorporating additional information as needed.

Due to the small and independent components, we adopt Micro-services Architecture (MSA) [10] which is an architectural style that complements "'loosely coupled"' multiple services. Since micro-services are lightweight software components with heterogeneous technologies, the communication technology in MSA is REST (HTTP based protocol). The MSA has a REST API structure and an API gateway. We leverage an API gateway for routing API services, ensuring API security (authentication, authorization), and optimizing API performance (load balancing, caching and proxying). The gateway service allows incoming HTTP requests to be routed to multiple back-end micro-services this is important to provide a unified API to consumers of our back-end resources, it provides load balancing, a traffic limiting mechanism, and the circuit breaker is configured to isolate the fault to ensure system reliability. To an external application, our API appears as a unified API surface.

Micro-services are implemented according to service specifications using the Java micro framework Spring Boot, and service interface endpoints are provided that consumers can run and invoke.

As per Figure 4, the proposed interconnection between fog nodes and SUMO simulation tool (which simulates IoT objects) follows a client-server architecture implementation. We chose Message Queuing Telemetry Transport (MQTT) [11] as the candidate IoT protocol for our implementation. MQTT is a publish/subscribe messaging protocol based on the TCP / IP protocol, it can be accessed in environments with limited devices (low processing power, low available bandwidth and low power consumption. Therefore, MQTT Broker manages all MQTT message traffic from the gateway. This allows to define and restrict the rules for reading and writing in topics.This way, services share information only with the devices they represent. Considering the characteristics of free, open and low RAM consumption software, the MQTT broker used in this work was the Mosquitto. We developed a middleware (Fog node-SUMO) in Java programming language serving as an intermediary tool between the MQTT Broker and SUMO. The fog node functions as a client, while the middleware operates as a server. It integrates the TraCI4J library, a Java-based implementation of TraCI, which is responsible for communicating with SUMO.

V. SIMULATION RESULTS AND ANALYSIS

We evaluated the proposed system by implementing and simulating it within a selected area that served as our

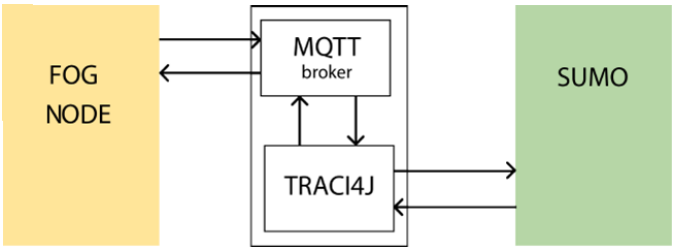


Figure 4. Interconnection between SUMO and the Fog node

case study: the CityMobil model [12], which simulates an airport-like parking environment. The model consists of 160 parking spots divided among 9 sectors: A, B, C, D, E, G, H, and I. Sectors A and I contain 10 spots each, while the remaining sectors have 20 spots each.

We compare the performance of applications based on our Fog infrastructure, compared to typical cloud implementation. The cloud simulation environment was implemented using Cloud Sim (Toolkit for Modeling and Simulation of Clouds) [13]. CloudSim provides users with ways to model and simulate execution cloud-based services. Therefore, by extending the base entities in the original simulator (data center, host, storage, and Cloudlet), it was possible to build a simulation backbone for the Cloud scenario. A set of functionalities can be associated with each entity (for example, CPU, RAM capacity, uplink network bandwidth), which provide ingenious measurements to use for final comparisons. In the cloud-based scenario, the cloud servers receive a continuous update from the parking lot detection infrastructure. Upon receipt of a request from the client application, the services that are deployed on the Cloud respond by providing the appropriate and nearest available parking space for the vehicle.

For the simulation, we conducted tests on monitoring devices with five different configurations. The simulations evaluated latency, network usage, and RAM consumption. The communication between the IoT node and its corresponding fog node is assumed to be through IEEE 802.15.4, or NB-IoT, or ZigBee, in which the transmission rates are 250 Kbps. The link rates between fog nodes in one domain are 100 Mbps and the link rates on the path from fog nodes to cloud servers are 10 Gbps. To obtain realistic values for the processing ratio, we looked at the processing capabilities of the ESP32 microcontroller (example of an IoT node) and an Intel dual-core i7 CPU (example of a fog node). Therefore, we assume that the average processing time of the IoT node is 30 ms. The comparison between fog and cloud computing average response delay between the end user and cloud server is shown in Figure 5 with various available parking spots from 1 to 100. The findings depicted in the simulation results show that the end-to-end network latency was reduced when relaying on our Fog approach in comparison with a cloud-based strategy. In fact, When the number of available parking spots increases, the number of fog nodes increases; therefore, sufficient computing

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.

power is available to efficiently process sensor data for that area and update the availability of parking spots more quickly. On the other hand, in the case of the cloud, cloud data centers are a bottleneck for simulation data. That's why, the disparity between the two approaches diminished as fewer parking spots became available.

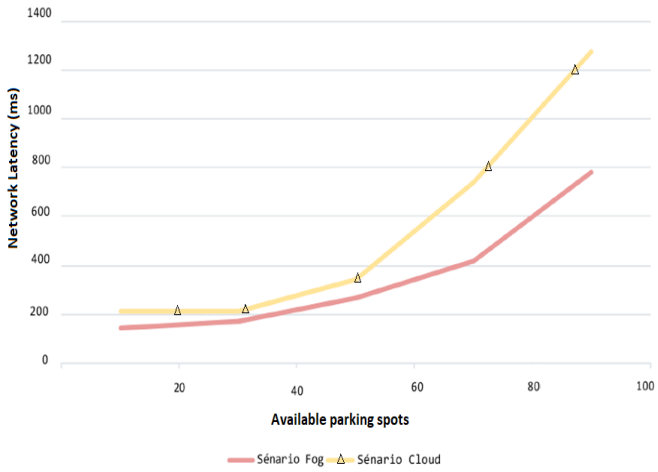


Figure 5. Fog computing and cloud computing comparison for average response delay

Figure 6 shows the network usage in fog and cloud computing environments with various available parking connected sensors from 3 to 30. It can be seen that as we increase available parking connected sensors, the network usage increases. In the cloud environment, network usage increases because all data from connected sensors are processed sequentially by a single cloud server upon connection. In contrast, in a fog environment, each parking area has its dedicated fog node, ensuring that sensors deployed in that area are processed locally, minimizing network overhead.

VI. CONCLUSION AND FUTURE WORK

In this paper, we have proposed an architecture for smart parking systems hosted on the Fog computing platform based on SSN ontology. The results demonstrate that the proposed fog-based architecture significantly reduces latency and network usage compared to cloud-based approaches. Fog computing's advantage lies in reducing frequent data transmissions to the cloud, thereby improving Quality of Service (QoS). This enhancement enables drivers to locate parking slots quickly, reducing both time and fuel consumption.

For the future works, we plan to experiment with a large scale of the proposed framework with increasing the number of parking areas, which probably will require investigating the load balancing issues on fog nodes to maintain the efficiency.

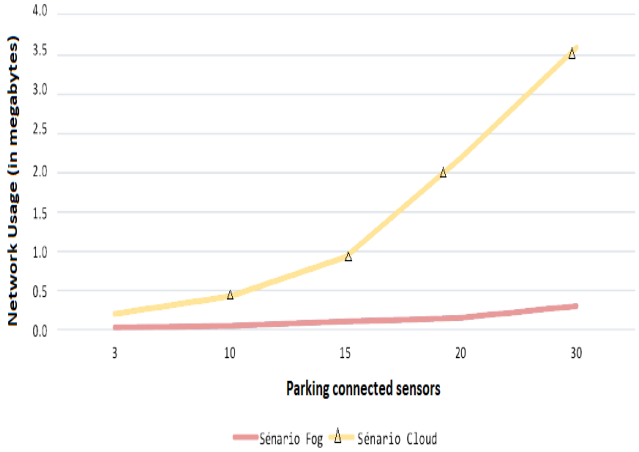


Figure 6. Comparing Network Usage between Fog Computing and Cloud Computing

REFERENCES

[1] T.-A. N. Abdali, R. Hassan, A. H. M. Aman, and Q. N. Nguyen, "Fog computing advancement: Concept, architecture, applications, advantages, and open issues," *IEEE Access*, vol. 9, pp. 75 961–75 980, 2021.

[2] M. Balfaqih, W. Jabbar, M. Khayyat, and R. Hassan, "Design and development of smart parking system based on fog computing and internet of things," *Electronics*, vol. 10, no. 24, 2021.

[3] R. Tandon and P. K. Gupta, "Optimizing smart parking system by using fog computing," in *Advances in Computing and Data Sciences*, M. Singh, P. Gupta, V. Tyagi, J. Flusser, T. Ören, and R. Kashyap, Eds. Singapore: Springer Singapore, 2019, pp. 724–737.

[4] Y. Zhang, C.-Y. Wang, and H.-Y. Wei, "Parking reservation auction for parked vehicle assistance in vehicular fog computing," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 4, pp. 3126–3139, 2019.

[5] K. Awaisi, A. Abbas, M. Zareei, H. A. Khattak, M. U. S. Khan, M. Ali, I. Ud Din, and S. Shah, "Towards a fog enabled efficient car parking architecture," *IEEE Access*, vol. PP, pp. 1–1, 11 2019.

[6] C. Tang, X. Wei, C. Zhu, W. Chen, and J. J. P. C. Rodrigues, "Towards smart parking based on fog computing," *IEEE Access*, vol. 6, pp. 70 172–70 185, 2018.

[7] S. Kumara, N. H. Prasad, M. Monika, H. Tuli, S. Supreeth, and S. Rohith, "Smart vehicle parking system on fog computing for effective resource management," in *2023 International Conference on Applied Intelligence and Sustainable Computing (ICAISC)*, 2023, pp. 1–6.

[8] M. Compton, P. Barnaghi, L. Bermudez, R. García-Castro, O. Corcho, S. Cox, J. Graybeal, M. Hauswirth, C. Henson, A. Herzog, V. Huang, K. Janowicz, W. D. Kelsey, D. Le Phuoc, L. Lefort, M. Leggieri, H. Neuhaus, A. Nikolov, K. Page, A. Passant, A. Sheth, and K. Taylor, "The ssn ontology of the w3c semantic sensor network incubator group," *Journal of Web Semantics*, vol. 17, pp. 25–32, 2012.

[9] A. Kusari, P. Li, H. Yang, N. Punshi, M. Rasulis, S. Bogard, and D. J. LeBlanc, "Enhancing sumo simulator for simulation based testing and validation of autonomous vehicles," in *2022 IEEE Intelligent Vehicles Symposium (IV)*, 2022, pp. 829–835.

[10] M. Söylemez, B. Tekinerdogan, and A. Kolukısa, "Challenges and solution directions of microservice architectures: A systematic literature review," *Applied Sciences*, vol. 12, p. 5507, 05 2022.

[11] R. Venanzi, B. Kantarci, L. Foschini, and P. Bellavista, "Mqtt-driven sustainable node discovery for internet of things-fog environments," in *2018 IEEE International Conference on Communications (ICC)*, 2018, pp. 1–6.

[12] A. Van Haare Heijmeijer and G. Vaz Alves, "Development of a middleware between sumo simulation tool and jacamo framework," *ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal*, vol. 7, no. 2, p. 5–15, Jun. 2018.

[13] W. Long, L. Yuqing, and X. Qingxin, "Using cloudsim to model and simulate cloud computing environment," in *2013 Ninth International Conference on Computational Intelligence and Security*, 2013, pp. 323–328.

Authorized licensed use limited to: National College of Ireland. Downloaded on January 19,2026 at 16:05:29 UTC from IEEE Xplore. Restrictions apply.